Composed table cell content

DocumentTableCell.node(DocumentNode) accepts any registered canonical node — paragraphs (with markdown), nested lists, even sub-tables — and the table layout pipeline prepares each child against the cell's resolved inner width, sizes the row from the prepared height, and dispatches the child's fragments through the standard NodeDefinition path.

1. Paragraph in cell with markdown rich text

Headline	Risk note
Q3 results were **strong** — *revenue* grew 18% year-over-year, driven by **enterprise renewals** and a *new* mid-market motion. Margin expanded by 4 points.	*Note:* the **EU rollout** is at risk if the *compliance review* slips past the *Nov 28* deadline. Owner: **Legal**.

2. Nested list in cell

A ListNode placed via DocumentTableCell.node(...) renders inside the cell's content area with the same depth cascade as a top-level nested list. The row sizes itself to the list's measured height.

Actions	Owners

- Approve EU rollout plan
  * Review compliance checklist
  * Confirm vendor contracts
- Schedule Q4 board update
	• **Legal** — EU compliance
	• **Finance** — Q4 forecast
	  - Reconcile renewal pipeline
	  - Ship board deck draft

3. Mixed composed and plain-text cells in the same row

Plain-text cells continue to use the existing DocumentTableCell.text(...) factory and render through the line-iteration code path. Composed cells render via NodeDefinition recursion alongside.

Status	Notes	Owner
On track	All P0 stories merged. *Buffer* for QA *holds* through **release branch cut**.	Eng
At risk	Compliance review tracking *one week* late. **Mitigation**: parallel scope reduction.	Legal

4. Composite nodes in a cell

A SectionNode or RowNode owns its children rather than drawing them itself, so a composed cell lays out the child's whole sub-tree: the section stacks its paragraphs, the row splits its band.

Section in a cell	Row in a cell
Renewal readiness Contracts countersigned, pricing locked, handover deck in review.	**Owner** Legal **Due** Nov 28